Contents

Cloud Platforms — AWS Deep Dive + GCP/Azure Overview **1**
1. AWS Core Services for DevOps . . . 1
Compute . . . 1
Networking . . . 1
Storage . . . 2
Databases . . . 3
IAM — Identity and Access Management . . . 3
Security Services . . . 4
Monitoring & Observability . . . 5
Cost Optimization Framework . . . 5
2. GCP Key Services for DevOps . . . 6
3. Azure Key Services for DevOps . . . 6
 Resources . . . 6

Cloud Platforms — AWS Deep Dive + GCP/Azure Overview

1. AWS Core Services for DevOps

Compute

EC2

```
Instance types:
  t3/t4g    — Burstable (dev, low-traffic)
  m6i/m7i   — General purpose (production apps)
  c6i/c7i   — Compute-optimized (CI agents, encoding)
  r6i/r7i   — Memory-optimized (databases, caching)
  p3/g4     — GPU (ML training/inference)
  x2idn     — High memory (SAP, large DBs)

Purchasing:
  On-Demand    — pay per hour, no commitment
  Reserved (1/3yr) — 40-72% savings
  Savings Plans  — flexible, apply to any instance family
  Spot          — up to 90% off, can be interrupted (2-min notice)
  Dedicated Host — for licensing compliance
```

ECS vs EKS vs Lambda vs Fargate

Service	Best For	Management Overhead
ECS (Fargate)	Simple containerized apps, no K8s expertise	Low
EKS	Production K8s, complex workloads	High
EKS Fargate	K8s workloads, no node management	Medium
Lambda	Event-driven, short tasks (<15min)	Very Low
App Runner	Simple web apps from container/code	Very Low

Networking

VPC Architecture (Production Best Practice)

```
VPC: 10.0.0.0/16

Public Subnets (one per AZ):
  us-east-1a: 10.0.0.0/24   → Internet GW, NAT GW, ALB
  us-east-1b: 10.0.1.0/24
```

```
  us-east-1c: 10.0.2.0/24

Private Subnets (application tier):
  us-east-1a: 10.0.10.0/24  → EKS nodes, ECS tasks
  us-east-1b: 10.0.11.0/24
  us-east-1c: 10.0.12.0/24

Database Subnets (isolated):
  us-east-1a: 10.0.20.0/24  → RDS, ElastiCache
  us-east-1b: 10.0.21.0/24
  us-east-1c: 10.0.22.0/24

Key resources:
  Internet Gateway    → public subnet internet access
  NAT Gateway (per AZ) → private subnet outbound internet
  Route Tables        → separate per tier
  VPC Endpoints       → private access to S3, ECR, SSM (no internet needed)
```

Load Balancers

Type	Layer	Use Case
ALB (Application LB)	L7 (HTTP/HTTPS)	Microservices, path/host routing, WebSockets, gRPC
NLB (Network LB)	L4 (TCP/UDP/TLS)	High performance, static IPs, TCP passthrough, low latency
CLB (Classic LB)	L4+L7	Legacy only — don't use for new deployments
Gateway LB	L3+L4	Network appliances (firewalls, IDS/IPS)

Route53

```
Record Types: A, AAAA, CNAME, ALIAS (AWS-specific, works at zone apex)
Routing Policies:
  Simple          → single value
  Weighted        → percentage split (A/B testing, canary)
  Latency         → route to lowest-latency region
  Failover        → primary + standby with health check
  Geolocation     → by user location (compliance, localization)
  Geoproximity    → location + bias adjustment
  Multi-value     → multiple IPs, health-checked (not a replacement for LB)
```

Storage

S3

```
Storage Classes (cost/performance tradeoff):
  Standard            → frequent access
  Intelligent-Tiering → automatic tier selection (monitoring fee)
  Standard-IA         → infrequent access (retrieval fee, lower storage cost)
  One Zone-IA         → non-critical, single AZ
  Glacier Instant     → archives retrieved in ms
  Glacier Flexible    → 1-12hr retrieval, cheapest storage
  Glacier Deep Archive → 12-48hr, lowest cost

Key Features:
  Versioning          → keep all versions, recover from deletes
  Replication (CRR/SRR) → cross/same region replication
  Lifecycle policies  → auto-transition/expire objects
```

```
  Object Lock (WORM)  → compliance: prevent modification/deletion
  Event notifications → trigger Lambda, SQS, SNS on object events
  Pre-signed URLs     → time-limited access for clients
  Transfer Acceleration → CloudFront edge for uploads
```

EBS vs EFS vs FSx

Service	Type	Use Case
EBS gp3	Block, single-AZ	Boot volumes, databases
EBS io2	Block, high IOPS	High-performance databases
EFS	Managed NFS, multi-AZ	Shared storage, Lambda, ECS
FSx for Lustre	High-performance parallel FS	ML training, HPC
FSx for Windows	SMB, Windows-native	.NET apps, AD integration

---

Databases

RDS / Aurora

```
RDS Multi-AZ:
  Synchronous replication to standby
  Automatic failover in 60-120 seconds
  Standby not readable (not for read scaling — use read replicas)

Aurora:
  6 copies of data across 3 AZs (2 per AZ)
  Failover < 30 seconds (Aurora Replicas promote instantly)
  Aurora Serverless v2 — auto-scale capacity units
  Global Database — multi-region with < 1s replication lag
  Up to 15 read replicas with load balancing endpoint

Connection best practices:
  Use RDS Proxy for Lambda (manages connection pooling)
  Use cluster endpoint for writes, reader endpoint for reads
  Use IAM database authentication (no passwords)
```

DynamoDB

```
Key concepts:
  Partition key + Sort key → primary key
  Capacity modes: On-Demand vs Provisioned (with Auto Scaling)
  GSI (Global Secondary Index) → alternate query patterns
  LSI (Local Secondary Index) → same partition, different sort key
  DynamoDB Streams → change data capture → Lambda triggers
  TTL → auto-expire items
  DAX → in-memory cache (microsecond latency)

On-Demand Capacity: pay per request, no planning needed
Provisioned: set RCU/WCU, cheaper at predictable load

Table design principle:
  Single-table design — multiple entity types in one table
  Access patterns first, schema second (opposite of relational)
```

IAM — Identity and Access Management

Key Concepts

```
Identities: Users, Groups, Roles
Resources: S3 buckets, EC2, RDS, etc.
Policies: JSON documents defining permissions

Policy evaluation:
  1. Explicit Deny → always wins
  2. Explicit Allow → grants access
  3. Implicit Deny → default (deny if no allow)

Policy types:
  Identity-based    → attached to user/role/group
  Resource-based    → attached to resource (S3 bucket policy, SQS)
  SCP (Service Control Policy) → AWS Organizations, sets max boundary
  Permission Boundaries → max permissions a role can have
  Session Policies  → temporary, passed during AssumeRole
```

IAM Best Practices

```
✓ Root account: only for billing, enable MFA, no access keys
✓ Use IAM roles for EC2/Lambda/ECS (never access keys on instances)
✓ Least privilege: start with read-only, add permissions as needed
✓ Enable MFA for all human users
✓ Use AWS SSO/Identity Center for human access (federate with IdP)
✓ Audit with IAM Access Analyzer
✓ Use condition keys: aws:SourceIp, aws:MultiFactorAuthPresent, aws:RequestedRegion
✓ Rotate access keys regularly (better: eliminate them entirely)
```

IRSA Pattern (EKS)

```
1. OIDC provider on EKS cluster
2. ServiceAccount annotated: eks.amazonaws.com/role-arn: arn:aws:iam::ACCT:role/ROLE
3. Trust policy on IAM role:
   {
     "Condition": {
       "StringEquals": {
      "oidc.eks.region.amazonaws.com/id/OIDC_ID:sub": "system:serviceaccount:NAMESPACE:SA_NAME"
       }
     }
   }
4. Pod token projected → AWS SDK exchanges for STS credentials
```

Security Services

Service	Purpose
GuardDuty	Threat detection — analyzes CloudTrail, VPC Flow Logs, DNS logs
Security Hub	Aggregates findings from GuardDuty, Inspector, Macie, etc.
AWS Config	Compliance — tracks resource config changes, evaluates rules
Inspector	Vulnerability scanning for EC2, ECR images, Lambda
Macie	S3 data classification — finds PII, sensitive data
KMS	Key management — envelope encryption for all AWS services
Secrets Manager	Managed secrets with auto-rotation
WAF	Layer 7 firewall — protects ALB, CloudFront, API Gateway

Service	Purpose
Shield	DDoS protection (Standard = free, Advanced = $3000/mo)
CloudTrail	API audit log — who did what, when, from where

Monitoring & Observability

```
CloudWatch Metrics:
  Built-in metrics for all AWS services
  Custom metrics via PutMetricData API or CloudWatch Agent
  High-resolution: 1-second granularity (extra cost)

CloudWatch Logs:
  Log groups + log streams
  Metric filters → create metrics from log patterns
  Log Insights → SQL-like query language
  Subscription filters → stream to Lambda, Kinesis, OpenSearch

CloudWatch Alarms:
  Threshold (static) or Anomaly Detection (ML-based)
  Actions: SNS, Auto Scaling, EC2 recover, Systems Manager

X-Ray:
  Distributed tracing
  Service map visualization
  Sampling rules to control cost

CloudTrail:
  API activity logging
  Management events (free), Data events (S3 object ops — paid)
  Insight events — unusual API call rate detection
```

Cost Optimization Framework

```
1. Right-sizing:
   AWS Compute Optimizer — ML-based recommendations
   Remove idle resources (unattached EBS, unused EIPs, idle LBs)

2. Pricing models:
   Savings Plans (Compute or EC2) — 1 or 3 year, up to 72% savings
   Reserved Instances — for stable workloads
   Spot Instances — stateless/fault-tolerant workloads

3. Architectural:
   S3 Intelligent-Tiering for variable access patterns
   Lambda for infrequent workloads vs always-on EC2
   CloudFront to reduce data transfer costs
   VPC Endpoints to avoid NAT Gateway data processing charges

4. Monitoring:
   Cost Anomaly Detection — ML-based alerts
   AWS Budgets — alerts on spend thresholds
   Cost Explorer — usage patterns, RI recommendations
   Resource tagging for chargeback/showback
```

---

2. GCP Key Services for DevOps

```
Compute:
  GCE (VMs) — equivalent to EC2
  GKE (Google Kubernetes Engine) — best managed K8s (Google invented K8s)
  Cloud Run — serverless containers (pay per request)
  Cloud Functions — FaaS

Networking:
  Global VPC (single VPC spans all regions — unique to GCP)
  Cloud Load Balancing — global anycast, single IP worldwide
  Cloud CDN
  Cloud DNS

Storage:
  GCS (Google Cloud Storage) — equivalent to S3
  Filestore — managed NFS
  Persistent Disk — block storage

DevOps:
  Artifact Registry — container/package registry
  Cloud Build — serverless CI/CD
  Cloud Deploy — managed CD to GKE
  Anthos — multi-cloud K8s management

GKE Autopilot:
  Google manages node pools, autoscaling, security
  Pay per pod resources, not nodes
  Built-in security hardening
```

---

3. Azure Key Services for DevOps

```
Compute:
  VMs, AKS (Azure K8s Service), Azure Container Apps, Functions

Networking:
  VNet, Azure Load Balancer, Application Gateway, Azure Front Door

Storage:
  Blob Storage, Azure Files, Managed Disks

DevOps:
  Azure DevOps (Pipelines, Repos, Boards, Artifacts) — full DevOps platform
  Azure Container Registry (ACR)

Identity:
  Azure AD / Entra ID — enterprise identity, MFA, Conditional Access
  Managed Identity — equivalent to AWS IAM roles for instances

Key Differentiators:
  Best Microsoft/Windows integration
  Azure DevOps widely used in enterprises
  Hybrid cloud with Azure Arc
```

🔗 Resources

- https://aws.amazon.com/architecture/ — AWS Architecture Center
- https://docs.aws.amazon.com/ — AWS Official Docs

- https://www.wellarchitectedlabs.com/ — Hands-on WAF labs
- https://cloud.google.com/docs — GCP docs
- https://learn.microsoft.com/en-us/azure/ — Azure docs
- https://www.awsgeek.com/ — Visual AWS cheatsheets
- https://github.com/open-guides/og-aws — Community AWS guide